

Como usar os *mockups*.

Este documento explica a estrutura do projeto, onde ficam os dados de placeholder, como substituí-los pelo seu backend real, e os prompts prontos para trabalhar com IA no redesign.

`CineTrack.html` – entry point

`src/data.jsx` – placeholders

`uploads/` – .razor originais

src/primitives.jsx – design tokens

Onde está cada arquivo

O projeto tem duas camadas: os **.razor originais** que você escreveu e o **redesign HTML** navegável gerado a partir deles.

CineTrack / – raiz do projeto

CineTrack.html **ENTRY POINT** Abrir no browser – carrega tudo

src/ Redesign HTML (protótipo)

└ **data.jsx** **PLACEHOLDERS** Todos os dados falsos ficam aqui

└ **icons.jsx** Ícones SVG inline

└ **primitives.jsx** **TOKENS**

Paleta, fontes, PhoneFrame,
MediaCard

- | screen-home.jsx Tela Home (variantes A e B)
- | screen-search.jsx Tela de busca com filtros
- | screen-detail.jsx Detalhe de filme/série
- | screen-stats.jsx Dashboard de estatísticas
- | screen-sync.jsx Sync com 4 abas
- | app.jsx Composição final + DesignCanvas + Tweaks

uploads/ Seus .razor originais (referência)

- | Main.razor ORIGINAL Shell + navegação
- | Home.razor ORIGINAL
- | Search.razor ORIGINAL
- | Detail.razor ORIGINAL

└ Stats.razor ORIGINAL

└ Sync.razor ORIGINAL

└ MediaCard.razor ORIGINAL Componente de card

Onde ficam e o que trocam

Todos os dados fictícios do protótipo estão em `src/data.jsx`. Edite esse único arquivo para testar cenários reais sem tocar nas telas.

`src/data.jsx` → `MEDIA[]`

Array de filmes e séries com 12 itens de exemplo

PLACEHOLDER

poster / backdrop

URLs de imagem do TMDb — formato

`https://image.tmdb.org/t/p/w500/{path}`

pelo campo da sua API (`ImageUrl` ou `ImagemExibicao`).

```
poster: POSTER('/lFf6LLrQjYldcZIitz0kGmMMi  
// ↓ no seu código:
```

```
poster: midia.ImagemUrl ?? midia.ImagemEx
```

status

Valores válidos: `'Assistindo'`,
`'Assistido'`, `'Pendente'`,
`'Abandonado'`. Devem bater com os
`StatusMidia` do seu enum.

imdb / tomato /
popcorn

Ratings públicos — `imdb` é decimal (ex:
`8.7`), `tomato` / `popcorn` são inteiros
percentuais (ex: `95`). Correspondência
direta com `ImdbRating`, `Tomatometer`,
`Popcornmeter` da sua VM.

progress

Objeto `{ watched: 14, total: 19 }` — só presente em séries em andamento. Mapeia para `EpisodiosAssistidos` / `TotalEpisodios`.

user

Nota do usuário em estrelas (1–5). `null` se não avaliado. Corresponde a `EstrelaUsuario`.

src/data.jsx → STATS{}

Objeto de estatísticas do usuário — usado em StatsScreen

PLACEHOLDER

horasAssistidas

Total de horas assistidas (inteiro). Exibido como hero no variant B. Vem do seu `EstatisticasVM.HorasAssistidas`.

generos[]

Array `{ name, count, pct }` — distribuição de gêneros. Precisa ser calculado no backend ou derivado dos gêneros das mídias. Não existe ainda na sua VM.

```
generos: [  
  { name: 'Drama', count: 42, pct: 38 },  
  { name: 'Sci-Fi', count: 28, pct: 25 },  
  ...  
]
```

atividade[]

12 pontos mensais de atividade `{ m: 'Mai', v: 8 }` para o gráfico de barras. Precisar de endpoint próprio.

topRated[]

Array de IDs das mídias com maior nota do usuário. Resolvido localmente via `FIND_MEDIA(id)`.

src/data.jsx → SYNC_DATA{}

PLACEHOLDER

Estado de sincronização — usado em SyncScreen (4 abas)

pendentes[]

Capturas aguardando confirmação. Cada item tem **capturado** (título bruto), **match** (título do banco), **confianca** (0–100) e **releases[]**. Mapeia para **AssociacaoPendenteVM**.

logs[]

Array de logs: **{ tipo: 'sucesso' | 'erro' | 'aviso' | 'info', data, msg }**. Vem de **LogCaptura**.

diagnostico[]

Resultados do parser: **claridade** (muito-claro / claro / moderado /

ambiguo), `score`, `modo`, `resumo` .

Mapeia para `DiagnosticoParserVM` .

Interagindo com o protótipo

1 Abra CineTrack.html no browser

O design canvas carrega automaticamente com todas as telas lado a lado. Use scroll e pan (arrastar o fundo) para navegar entre as seções **A · Conservador** e **B · Ousado**.

2 Abra qualquer artboard em fullscreen

Passe o mouse sobre um frame de celular — aparece o ícone  no canto superior direito. Clique para abrir fullscreen. Use   para navegar entre artboards e  para fechar.

3

Interaja com os frames

Toque qualquer card de mídia para abrir o **Detail**. A bottom nav alterna entre Home, Search, Stats e Sync. Na tela Sync, as 4 abas (Status, Pendências, Logs, Diagnóstico) são clicáveis.

4

Use o painel de Tweaks

Clique no botão **Tweaks** no canto inferior da tela. Alterne a **densidade** entre Confortável e Compacto — o layout de todos os frames atualiza em tempo real.

5

Edite os placeholders

Abra `src/data.jsx` e substitua qualquer item do array `MEDIA` ou do objeto `STATS`. Recarregue o browser — nenhum build necessário.

Como pedir alterações com IA

Cole estes prompts diretamente — eles já têm contexto suficiente pra qualquer IA entender a estrutura do projeto.

TROCAR PLACEHOLDER POR DADO REAL

COPIAR

No projeto CineTrack, o arquivo `src/data.jsx` contém dados de placeholder. Quero substituir o array `MEDIA[]` pelo seguinte dado real da minha API:

[cole aqui o JSON retornado pela sua API]

Mapeie os campos assim:

- `Titulo` → `title`
- `TituloOriginal` → `original`
- `Ano` → `year`
- `Tipo` (enum: Filme=0, Serie=1) → `type` ("filme" ou "serie")
- `ImdbRating` → `imdb`
- `Tomatometer` → `tomato`
- `Popcornmeter` → `popcorn`
- `EstrelaUsuario` → `user`
- `StatusDescricao` → `status`
- `ImageUrl ?? ImagemExibicao` → `poster`
- `EpisodiosAssistidos` / `TotalEpisodios` →
`progress.watched` / `progress.total`

Mantenha a estrutura do arquivo idêntica; troque apenas os dados.

No projeto CineTrack, quero adicionar uma nova tela chamada **[nome da tela]**.

Contexto do projeto:

- Entry point: `CineTrack.html`
- Telas existentes: Home, Search, Detail, Stats, Sync (em `src/screen-*.jsx`)
- Cada tela é um componente React com props `variant` ("a" ou "b") e `density` ("comfortable" ou "compact")
- Design tokens ficam em `src/primitives.jsx` → objeto `THEME`
- Componentes compartilhados: `PhoneFrame`, `BottomNav`, `MediaCard`, `SectionHeader`, `Icon`

Crie o arquivo `src/screen-[nome].jsx` seguindo o mesmo padrão das telas existentes. Depois:

1. Importe-o em `CineTrack.html` como `<script type="text/babel" src="src/screen-[nome].jsx"></script>`
2. Adicione o render no componente `Phone` em `src/app.jsx`
3. Adicione um ícone na `BottomNav` em `src/primitives.jsx`
4. Adicione dois `DCArtboard` (variant A e B) nas seções do DesignCanvas em `src/app.jsx`

O conteúdo da tela deve ser: **[descreva aqui o que a tela faz]**

AJUSTAR VISUAL DE UMA TELA

COPIAR

No projeto CineTrack, quero ajustar a tela **[nome: Home / Search / Detail / Stats / Sync]**.

O arquivo é `src/screen-[nome].jsx`.

Há dois variants: `variant="a"` (conservador) e `variant="b"` (ousado/editorial).

Tokens de design disponíveis no objeto `THEME` (em `src/primitives.jsx`):

- Cores: `THEME.gold` (#e8b94a), `THEME.red` (#c64133),
`THEME.bg`, `THEME.bg1`, `THEME.bg2`, `THEME.text`,
`THEME.text2`, `THEME.text3`
- Fontes: `THEME.serif` (Fraunces), `THEME.sans` (Inter),
`THEME.mono` (JetBrains Mono)

Quero que você: **[descreva a mudança desejada]**

Altere apenas o variant **[a / b / ambos]**. Mantenha o restante idêntico.

ADICIONAR TWEAK

COPIAR

No projeto CineTrack, quero adicionar um novo controle no painel de Tweaks.

O painel fica em `src/app.jsx` no componente `App`, usando o `TweaksPanel` e `useTweaks()`.

Tweaks existentes:

- `density`: "comfortable" | "compact" → passado via prop para todos os `Phone`

Quero adicionar o tweak: **[descreva o tweak: nome, tipo (toggle/radio/slider/select), opções possíveis e o que controla visualmente]**

Passos:

1. Adicione o default no objeto `TWEAK_DEFAULTS`
2. Adicione o controle no JSX do `TweaksPanel`
3. Passe o valor como prop para o componente `Phone`
4. Faça o componente de tela relevante consumir a prop e mudar o visual



Tokens e paleta

Todos os valores ficam no objeto `THEME` em `src/primitives.jsx`. Edite lá para mudar o visual globalmente.



Superfícies

`bg` · `bg1` · `bg2` · `bg3` —

camadas warm-black



Accent

`gold` · `red` · `success` ·

`warning`

Fraunces

JetBrains Mono

`THEME.serif` — títulos,
displays, hero numbers. Elegância
editorial.

`THEME.mono` — metadados,
ratings, labels técnicos,
contadores.



Para mudar a cor de accent do app inteiro (de dourado para, ex., azul), edite apenas `THEME.gold` em `src/primitives.jsx`.
Todos os componentes herdam o valor.

✓ Os pôsteres vêm de `image.tmdb.org/t/p/w500/` — substitua
pelo seu CDN trocando a função `POSTER()` no topo de
`src/data.jsx`.

